

D4 – AI Usage Journal (Part 2 continuation)

Recipix v4.1 – CSE 341 Semester Project

Berk Hakan Öge (210104004132)

23 May 2026

Part 2 (entries 16–24)

Continues my Part-1 D4 journal (210104004132_D4_P1.pdf, 15 entries / 6 sessions, May 6–8). This document documents **9 entries from 8 sessions** during Part 2 work, dated May 20–23. The professor session (13) is cited by two entries because it was used across two days; every other session has one entry.

#	Session	Tool / Model	Date
7	Web Planning B – Part 2 Roadmap and Pair-Split	Claude.ai web (Sonnet)	2026-05-20
8	Web Experiment E1 – EBNF generation (late catch-up)	Claude.ai web (Opus 4.7, thinking on)	2026-05-21
9	Web Experiment E2 – Type equivalence	Claude.ai web (Opus 4.7, thinking on)	2026-05-21
10	Web Experiment E3 – AI implements <code>_check_ActionStmt</code>	Claude.ai web (Opus 4.7, thinking on)	2026-05-21
11	Developer Agent – Interpreter + Rendering build	Claude Code (Sonnet 4.6, autonomous)	2026-05-21
12	PM Agent – Interpreter Commits Review	Claude Code (Opus 4.7)	2026-05-21
13	Professor Agent – Pass-1 + Final Rubric Review	Claude Code (Opus 4.7)	2026-05-21
14	Tester Agent – Verification (two turns)	Claude Code (Opus 4.7)	2026-05-22

The three Claude Code transcripts are submitted alongside this PDF as JSONL files:

- `session11_developer_interpreter_2026-05-21.jsonl`
- `session12_pm_interpreter_review_2026-05-21.jsonl`
- `session13_professor_2026-05-21_to_2026-05-23.jsonl`
- `session14_tester_2026-05-22_to_2026-05-23.jsonl`

The four web sessions have shareable URLs cited inline within each entry.

1. Session 7 – Web Planning B: Part 2 Roadmap (Claude.ai Web)

Source session: `claude.ai/share/fb1553f4-e7c3-4913-819f-521c6219960d`

Single Claude.ai web chat used to plan the Part 2 split with my partner, generate the contract proposal doc to send to İsmail, and surface the design decisions that needed locking before either of us wrote new code. Four substantive turns, treated as one entry because they form one continuous planning arc.

Entry #16 of 24 • Date 2026-05-20 **• Phase** Project planning / pair split **• AI tool** Claude.ai web (Sonnet)

Goal. Take Part 1’s submitted state into a Part 2 plan İsmail and I could both sign off on before either of us started writing code. Specifically: pick a pair split, lock the integration contracts that would let us work in parallel, and produce a markdown doc I could send to İsmail rather than walking him through chat scrollbar.

Prompt. Started with *"We are building our own programming language. Here the github repo for it. We have completed Part1 of the project. You can read the project handout.pdf file for details about the project. Read through the files and inspect the work done overall for the part1. After you are done with learning the project we will start planning on what to do next for part2."* Then three follow-up turns: ask for split options, pick Option A, ask for the markdown doc to share with İsmail.

Response (key part). The model pulled the spec, parser source, AST, error module, lexer contract, fixtures, my D5, İsmail’s D5, the D6, and the handout. Three things it produced were load-bearing: (1) a clean read of where Part 1 actually shipped, including the three deferred items I’d flagged in my D5 (*AmbiguousCall* resolution, *evaluate*-only constraint, single-return rule) and which Part 2 phase they each landed in. (2) Three pair-split options, each with a Sebesta-chapter argument; recommended Option A (Berk = interpreter, İsmail = type checker) was justified by İsmail’s stated D5 intention and by my ownership of the AST shapes. (3) A full Phase-0 contract list (TypedAST contract, runtime value dataclasses, action-verb signature table, quantity-normalization rule, environment-chain shape, file layout, sample-output agreement) plus a draft `part2_plan.md` written in my voice.

Accepted. The Option-A pair split; the in-place TypedAST annotation contract; the runtime value dataclasses as drafted (*RtQuantity*, *RtPinch*, *RtIngredient*, *RtRecipe*, *RtList*); the quantity-normalization rule (type checker tracks dimension, interpreter normalizes to base units at *QuantityLit* evaluation); the shared `environment.py` design. The first draft of `part2_plan.md` became the working contract; subsequent revisions (Rev 2, Rev 3) were edits in a separate Claude Code session, but the bones came from here.

Rejected / modified. The 12-row per-verb action-verb signature table was the model’s straw proposal; I collapsed it to three signature classes (Heterogeneous, Homogeneous, Nullary) in `part2_plan.md` §2.3 because the per-verb form was unjustified branch explosion in the checker. The model accepted the collapse without argument when challenged later – see Session 10 / Entry 19.

Errors caught. One soft error: the model’s first read called the action-verb signature table “small but exam-bait” and proposed twelve rows. The right answer was three. I caught it by reading the §2.3 list against the spec’s decision #29 (“action verbs are an exception to the homogeneous-list rule”) – twelve per-verb signatures don’t honor a single-decision carve-out; three signature classes do. The model’s later proposal had the same blind spot. This is the failure mode I now look for first when prompting any AI on design decisions: it expands tables in the direction of “more rules cover more cases” rather than collapsing them in the direction of “fewer rules cover the same cases at lower exam and maintenance cost.”

Reflection. This session’s contribution was that it forced me to articulate the Part 2 split out loud before talking to İsmail. The chapter mapping (Ch. 6 = İsmail, Ch. 7 = Berk) is the line I rehearsed for the meeting. The runtime contract doc is what made the parallel build possible the next day. The model was best at *briefing-from-spec* (it correctly identified the three deferred items from my Part 1 D5 without me prompting), worst at design rationale that has to be in my own voice – every D1 [P2] paragraph it offered, I rewrote. Lesson: I use the AI to surface decisions I should make, never to make them.

2. Session 8 – Web Experiment E1: EBNF Generation

Late-execution acknowledgment. Experiment E1 was required for the Part 1 D4 journal but was omitted from my submission by oversight. I noticed the gap on 2026-05-21 during a Part 2 D4 prep pass and am running E1 here, honestly dated, as a catch-up entry filed with my Part 2 D4 submission. This entry does **not** retroactively satisfy the Part 1 requirement; it documents the experiment for completeness and is noted in my D5 retrospective. The model used (Claude Opus 4.7) postdates Part 1’s working window, which is another reason to be explicit about dating rather than smuggle it into earlier dates.

Source session: claude.ai/share/6d67cab6-006e-414f-b05c-ada5fa5b0d22

Two-turn session: cold EBNF generation, then comparison against the locked spec.

Entry #17 of 24 • Date 2026-05-21 • Phase Design / EBNF retrospective • AI tool Claude Opus 4.7 (extended thinking: on)

Goal. Generate a first-pass EBNF grammar for Recipix from a short prose description and compare it against my locked spec (v4.1, §13), so I can see which language-specific decisions an AI derives on its own and which it silently smooths over into PL-textbook defaults.

Prompt. Verbatim, one paragraph – no productions or repository content leaked into turn 1; only the one-paragraph description of the language: “Recipix is a small DSL for writing parameterized recipes that can be scaled and have ingredients substituted. The primitive types are `int`, `float`, `bool`, five quantity types (`Mass`, `Volume`, `Count`, `Temperature`, `Duration`), and a separate `Pinch` type...” – continues through structured types, user-visible constructs, the three call-site operators, quantity literals, the unary `quantity_of` operator, and the no-chained-comparison constraint. Ends with “Please give me an EBNF grammar.”

Response (key part, turn 1). AI’s first-pass EBNF, generated cold. Highlights:

```
Expression      ::= LogicalOr
LogicalOr       ::= LogicalAnd ( "or" LogicalAnd )*
LogicalAnd      ::= Comparison ( "and" Comparison )*
Comparison      ::= Additive ( CompOp Additive )?      (* non-
    associative *)
CompOp          ::= "<" | "<=" | ">" | ">=" | "==" | "!="
Additive        ::= Multiplicative ( ( "+" | "-" ) Multiplicative )*
Multiplicative  ::= Unary ( ( "*" | "/" | "%" ) Unary )*
Unary           ::= ( "-" | "not" | "quantity_of" | "evaluate" ) Unary
    | Postfix
SubstituteExpr  ::= "substitute" "(" Expression "," Expression ","
    "with" ":" Expression ","
    "ratio" ":" Expression ")"
QuantityLit     ::= ( INT_LIT | FLOAT_LIT ) Unit
```

Commentary from the AI: quantity literals as *two tokens* “joined in the parser”; chained comparison “forbidden by making Comparison non-associative”; if/else “no dangling-else problem because bodies are braced”; `quantity_of` “sits at the unary layer”. The AI volunteered uncertainty about `Count` (“I left it without a dedicated unit on the assumption it’s promoted from INT_LIT by context”) and flagged it as the most likely divergence.

Response (key part, turn 2). After giving the AI the GitHub link to `docs/recipix_v4_1_spec.md`, its self-assessment: “I got the high-stakes structural calls right – two-token quantities, non-associative comparison, mandatory if/else braces, `quantity_of` at the unary level – but missed the substitute bare-identifier restriction, and several smaller surface details.”

Decision	AI cold	Locked spec
#7 quantity literals as two tokens	two tokens, parser-joined	match (AI missed <code>count</code> , <code>mg</code> , used wrong unit set)
#17 comparison non-associative	non-associative	match, same reasoning
#20 mandatory braces on if/else	mandatory	match
#34 <code>quantity_of</code> at unary	unary level, parens-free operand	“right floor, wrong storefront” – match on placement
#35 substitute slots as IDENT	all four slots <code>Expression</code>	miss – the one big design call the AI invented from prose was wrong

The AI’s framing of the substitute miss: *“Your AST node carries `original_name`: `str` and `replacement_name`: `str`, not `Expr` – there is no AST shape that pretends the slot could be computed. That’s the cleanest design lesson in the whole comparison – grammar-as-documentation-of-semantics, not just grammar-as-parser-spec.”*

Accepted. The four high-stakes calls match the locked grammar’s intent, and the AI’s reasoning on each lines up with the rationale I locked into the spec. The precedence ladder shape (seven levels in the right top-down order) and the program skeleton both match. Even the “right floor, wrong storefront” miss on `quantity_of` proves the type-system reasoning was sound; only the surface syntax differed.

Rejected / modified. The substitute-slot miss plus a cluster of over-permits where the spec locks down: optional `let` type annotation, `List<T>` as a user-writable type, top-level `IngredientDecl`, step body absent, no closed action-verb set, function bodies with no single-return restriction, postfix indexing, keyword logical operators (`and/or/not` instead of `&&/||/!`), wrong unit set.

Errors caught. (1) **Substitute slots as Expression instead of IDENT** – under the AI grammar, `substitute(r, flour + salt, with: milk, ratio: 1.0)` parses; under the spec it’s a parse error. (2) **Cluster of over-permits** where the spec locks down – predictable shape of “derive a grammar from prose without the locked decisions in view.” (3) **Unit set drift** – AI invented units (`°F`, `s`, `h`) and missed locked ones (`count`, `mg`).

Reflection. The thing that surprised me here is how much Opus got right cold with thinking on. Four of five big design calls came back match. What didn’t change is the *kind* of thing it still misses. Decision #28 (“ingredient identity is the binding”) drives decision #35 (substitute slots are bare identifiers), and there’s no way to derive #35 from prose unless you’ve already had the #28 conversation. The over-permits – `List<T>` annotations as user-writable, top-level ingredients, optional `let` type, indexing, multi-return function bodies – are the same shape: the AI admits whatever generic PL languages admit; the spec restricts what Recipix specifically restricts. *Lesson:* I can’t trust a “generate a grammar from a paragraph” pass to surface decisions whose rationale lives outside that paragraph. I have to drive the AI with the rationale already in hand. That’s also why I’m filing this entry on its real date and not backdated – the value of the experiment is in the comparison, and the comparison only holds if the dates and tool name are honest.

3. Session 9 – Web Experiment E2: Type Equivalence

Source session: claude.ai/share/dd9a44cf-584d-4194-9312-2b512752f90d

Two-turn session: cold equivalence recommendation, then comparison against the locked spec.

Entry #18 of 24 • Date 2026-05-21 • **Phase** Type checker / Design • **AI tool** Claude Opus 4.7 (extended thinking: on)

Goal. Test whether a fresh AI session, given only a one-paragraph description of Recipix and its structured types, can derive the type-equivalence rule I actually settled on in v4.1 – or whether it gives a textbook answer that misses the domain reason for the split.

Prompt. Verbatim, Recipix domain framing plus the four structured types (**Ingredient**, **Recipe**, **Step**, **List<T>**), asking for a per-type name-vs-structural recommendation plus a small example program that depends on the rule.

Response (key part, turn 1, AI’s first-pass recommendation):

Type	AI cold	AI’s justification (paraphrased)
Ingredient	name	“Domain noun... generic fields would collapse with hypothetical Cost/Allergen records.”
Recipe	name	“Top-level entity; accidental equivalence at the top of the type lattice is a worse failure mode than at the leaves.”
Step	name	“Optional fields make decisive – under structural rules you’d get four distinct shapes per optional-field combination.”
List<T>	structural in parameter, nominal in constructor	“Standard treatment for parametric types – ML/Haskell/Java generics converge here.”

The AI offered a brownies-recipe example using pseudo-syntax (**Step**("desc", **actions**: [...]) constructor calls inside a **steps**: [...] list literal) claiming the three Steps in the brownies recipe (one with neither modifier, one with both, one with only **duration**) would have “three distinct structural shapes” and fail to unify in a **List<Step>** under structural equivalence.

Response (key part, turn 2, comparison against the locked spec): Self-assessment:

1. **Recipe – match.** Same call, same reasoning.
2. **List<T> – match.** Same conceptual move, phrased differently.
3. **Ingredient – divergence, with self-correction.** *“My reasoning was forward-looking: avoid silent collisions with hypothetical Cost {name, quantity} or Allergen {name, quantity} types in some v2. That argument falls apart against your actual v1, because there is no user-defined record syntax – Ingredient is the only {name, quantity} record the type system will ever see, and §1 makes the type-name set closed. So my choice was buying safety against a threat that can’t exist in the language as specified. Pick the more permissive rule when over-restriction has no upside.”*
4. **Step – divergence on reasoning.** The AI realized its argument was based on the strict-Sebesta reading of structural equivalence (each ?-field present-or-absent creates a distinct shape) – *not* how the spec applies the rule. The spec treats optional fields as part of the *single* declared shape.

The AI’s one-line summary: *“I got Ingredient genuinely wrong (defensible reasoning, wrong call for a sealed v1), I got Step nominally wrong but on a question your spec doesn’t actually need to answer, and my example was sold as a discriminating test case when it isn’t one under your rules.”*

Accepted. Recipe and List<T> calls match the spec’s intent. AI’s framing in turn 2 – “pick the more permissive rule when over-restriction has no upside” – is a one-liner defense of decision #10’s structural-for-Ingredient call that I can quote almost verbatim in D1 §4.8.

Rejected / modified. Three substantive divergences: (1) Ingredient picked name (spec picks structural). (2) Step reasoning relied on strict-Sebesta reading; spec uses optional-fields-inclusive reading. (3) Pseudo-syntax in the example (constructor-call form, list-literal of steps) is not Recipix surface syntax.

Errors caught. (1) The AI’s domain reasoning optimized for extensibility scenarios that the closed v1 type-name set forecloses. (2) The AI assumed a stricter notion of structural equivalence than the spec uses. (3) The example program does not run under the rule it claimed to demonstrate – the literal test the handout’s E2 specification names.

Reflection. The interesting result here isn’t that the AI got Ingredient wrong – it’s that the reasoning behind the wrong answer was sophisticated and forward-looking, and would have been correct for a different language. It hedged against Cost {name, quantity} collisions that v1 makes impossible because the type-name set is closed. That’s the failure mode I think I’ll see most often when prompting AI on Recipix design questions: the AI optimizes for the kind of evolution a generic record-based DSL would undergo, and v1 has already foreclosed those evolutions. *Lesson:* the AI accelerates generating design options. It does not decide which option a sealed v1 has already committed to. That’s mine.

4. Session 10 – Web Experiment E3: AI Implements `_check_ActionStmt`

Source session: claude.ai/share/8190d922-7330-4181-84cd-40e1748c1e99

Two-turn session: cold implementation, then comparison against plan §2.3 and the typechecker stub.

Entry #19 of 24 • Date 2026-05-21 **• Phase** Type checker **• AI tool** Claude Opus 4.7 (extended thinking: on)

Goal. Have the AI implement `_check_ActionStmt` on the `TypeChecker` class – the §10 error-#13 enforcer for the twelve closed verbs in spec §8 – without first showing it the three-class table from plan §2.3, then compare its inferred signatures to the locked design.

Prompt. Verbatim: the twelve verbs (`combine`, `mix`, `add`, `sprinkle`, `pour`, `drizzle`, `whisk`, `blend`, `knead`, `melt`, `bake`, `flip`), the type-string contract for `env.lookup`, the four test inputs whose right answers I knew in advance, and the requirement to use at least one helper data structure (a verb-signature dict/table).

The AI flagged an unsolicited prompt ambiguity before answering: “*you said $D \in \{Mass, Volume, Count, Temperature, Duration\}$ – Pinch is not in D – but then the example return value mentions `Quantity<Pinch>`, and test #1 uses `Ingredient<Pinch>`*”. Real prompt bug; the spec treats `Pinch` as a primitive, not a dimension, but the implicit-projection rule (#31) and the test inputs effectively require the checker to handle `Ingredient<Pinch>` as well-formed.

Response (key part, turn 1, AI’s first-pass implementation):

```
VERB_SIGS = {
    'combine': ('variadic', [], _ingredient_of(), 2),
    'mix':      ('variadic', [], _ingredient_of(), 2),
    'blend':    ('variadic', [], _ingredient_of(), 2),
    'whisk':    ('fixed',    [_ingredient_of()]),
    'flip':     ('fixed',    [_ingredient_of()]),
    'knead':    ('fixed',    [_ingredient_of('Mass')]),
    'melt':     ('fixed',    [_ingredient_of('Mass')]),
    'pour':     ('fixed',    [_ingredient_of('Volume')]),
    'drizzle':  ('fixed',    [_ingredient_of('Volume')]),
    'sprinkle': ('fixed',    [_ingredient_of('Mass', 'Count')]),
    'add':      ('variadic', [_ingredient_of(), _quantity_of(), 1]),
    'bake':     ('fixed',    [_ingredient_of(),
                             _quantity_of('Temperature'),
                             _quantity_of('Duration')]),
}
```

Dispatcher distinguishes two signature kinds (`'fixed'` and `'variadic'`) with per-arg shape-check delegated to predicate factories. The AI explicitly named its biggest guesses up front: *“combine/mix/blend accept any combination of dimensions... bake is fixed-arity 3. This is the biggest guess in the table.”*

Response (key part, turn 1, predicted verdicts on the four tests):

#	Test input	AI verdict	AI reason
1	<code>combine(flour, salt)</code>	accept	variadic ≥ 2 , both Ingredients
2	<code>pour(flour:Mass)</code>	reject	pour locked to Volume
3	<code>bake(x)</code>	reject	bake arity 3, called with 1
4	<code>blend(milk:Vol, sugar:Mass)</code>	accept	blend in any-mixed-dim bucket

Response (key part, turn 2, comparison against plan §2.3): *“The verdict is uncomfortable but useful for your E3 entry – I diverged in three substantive ways, and two of four test predictions flip under §2.3.”* The three substantive divergences (AI’s words):

1. **Per-verb table vs three-class collapse.** *“I built a per-verb table. §2.3 collapses twelve verbs into three classes precisely to avoid the per-verb branch explosion. I optimized for cooking semantics; you optimized for type-system parsimony.”*
2. **Heterogeneous-group misses.** Partial credit on `combine/mix` (required ≥ 2 args instead of ≥ 1); `add` was bespoke (spec puts it in heterogeneous); `sprinkle` was restricted to Mass/Count (spec admits any dimension including Pinch).
3. **Homogeneous-group misses.** Most divergent. Locked per-verb dimensions; missed homogeneity rule entirely; missed `flip` as nullary. Also missed decision #31.

Four-test re-run under §2.3:

#	Test	AI cold	§2.3	Status
1	<code>combine(flour, salt)</code>	accept	accept	match (right verdict, wrong rule)
2	<code>pour(flour:Mass)</code>	reject	accept	flipped
3	<code>bake(x)</code>	reject (arity 3 $\neq$ 1)	reject (arity 0 $\neq$ 1)	match-by-coincidence
4	<code>blend(milk, sugar)</code>	accept	reject	flipped

AI’s score framing: *“If you’re scoring strictly on ‘did the AI produce the same accept/reject column as the spec,’ 2/4. If you’re scoring on ‘did the AI produce the right verdict for the right rule,’ 1/4.”* The AI then volunteered a §2.3-aligned correction unsolicited – a clean three-class dispatcher with sets + `_project_to_quantity` helper. That correction became a usable starting point for the implementation.

Accepted. Dispatcher skeleton is reusable. The `_parse_type` helper survives unchanged. AI’s `error_code=13` choice is consistent with the spec. The AI’s corrected `_check_ActionStmt` body in turn 2 became part of the eventual implementation. AI’s unsolicited prompt-ambiguity flag (Pinch in D vs not in D) surfaced something the v4.2 spec needs to clarify.

Rejected / modified. Everything in the cold first-pass except the dispatcher skeleton: the twelve-row `VERB_SIGS` table (replaced with the three-class form), per-verb dimension locks (no per-verb dim in spec, only homogeneity), `combine/mix/blend` min-arity ≥ 2 (spec is ≥ 1), bespoke `add` signature (spec heterogeneous), `bake` as fixed-arity 3 (spec nullary; temperature/duration moved to step modifiers), `flip` unary (spec nullary), `whisk` unary (spec homogeneous), missing implicit Ingredient $\rightarrow$ Quantity projection.

Errors caught. (1) Test 2 (`pour(flour)`) verdict flipped – AI rejected on hard-coded Volume; spec accepts because `pour` is homogeneous and single-arg is trivially homogeneous. (2) Test

4 (**blend**) verdict flipped – AI grouped with combine/mix as heterogeneous; spec puts in homogeneous with dimension mismatch. (3) Test 3 matched verdict for the wrong reason. (4) Decision #31 (Ingredient → Quantity projection) absent. (5) Per-verb cooking-semantics over-specify against the spec’s type-system parsimony.

Reflection. The biggest takeaway from this one is that the AI’s design instinct – “more rules cover more cases” – is exactly opposite to the spec’s instinct of “fewer rules cover the same cases at lower exam and maintenance cost.” Twelve per-verb signatures look like authority on first read; the AI’s own retrospective lands on this in one line: *“I optimized for cooking semantics; you optimized for type-system parsimony.”* That framing is the one I want in §4.8 and in my head for the exam. The other half of the experiment is what happened after the spec came into view: the AI wrote its own §2.3-aligned correction, unsolicited, and got it essentially right. **AI implements decisions well. It does not invent them well.** That’s the working pattern I’ll keep across the rest of Part 2 – I drive the decisions, the AI drives the implementation, and the journal is the record of what happens when those roles slip.

5. Session 11 – Developer Agent: Interpreter + Rendering Build

Source: `session11_developer_interpreter_2026-05-21.jsonl` (uploaded alongside this PDF). Autonomous developer-agent run dispatched from a Claude Code PM session, executed in a single long autonomous turn.

Entry #20 of 24 • Date 2026-05-21 • **Phase** Interpreter implementation (autonomous agent)
 • **AI tool** Claude Sonnet 4.6 (Claude Code agent)

Goal. Build the runtime half of Recipix Part 2: `interpreter.py` and `rendering.py`, plus a 36-test suite covering the §4 task list in `part2_plan.md`. The type checker remains stubbed (İsmail’s half); the agent must not dispatch `AmbiguousCall` and must fail loudly if it sees one. Sample 1 and Sample 2 must produce rendered cookbook-style output; Sample 3 must remain a type-error path.

Prompt. ~3.5k-character developer-agent prompt covering: scope (interpreter + rendering only; do not modify Part 1 files; do not touch `typechecker.py`), the runtime contract sections of `part2_plan.md` to read first, the 14-step recommended order, the runtime-error checklist (5 errors from spec §10), and the acceptance criteria.

Accepted. The agent produced commits `51c1b03` (interpreter + rendering implementation, 619 lines) and `fac696a` (36-test suite). Tools used (counted from the JSONL): 9 Edit/Write, 15 Read, 25 Bash for verification, plus 21 `TaskCreate` / `TaskUpdate` calls – the agent self-tracked its 14-step task list throughout and ticked items off as it implemented them. The eager-step-evaluation strategy was the one locked at the time in plan §2.2; the agent followed it. `AmbiguousCall` dispatch raises `RuntimeRecipixError` unconditionally – the loud-fail behavior asked for; integration tests use a separate helper to simulate the typechecker’s resolution.

Rejected / modified. Three findings from a follow-up PM review (entry 21) led to commit `d5c539c`: the original `AmbiguousCall` handler had a fallback dispatching via symbol-table `isinstance` checks (would let a missing-typechecker bug pass silently for any name in scope), and the test suite had gaps on runtime error #4 (`substitute` of unknown ingredient) and decision #31. All three were closed in the follow-up commit; the agent self-corrected on all three when given the gap report.

Errors caught. The PM review (entry 21) caught all three substantive gaps before any of them shipped to a passing-on-the-surface-but-broken-in-integration state. The agent itself ran clean – the 36 interpreter tests it wrote were green on the first full run, and the agent’s own report acknowledged no self-caught bugs during the build. Test-suite count progressed: 111

tests pre-build (the existing parser + regression baseline) → 147 after 51c1b03+fac696a (+36 interpreter tests) → 152 after the PM-fix follow-up d5c539c (+5 tests covering gaps F, G, I).

Reflection. The pattern I keep seeing with autonomous developer agents in Claude Code is that when the contract is locked tightly enough, the agent’s free movement is on the implementation surface only – it doesn’t drift into design decisions, it doesn’t invent abstractions I didn’t ask for, and it ticks the task list. *AI is good at breadth-first implementation when the spec is locked* – exactly the same finding as my Part 1 entries 14 and 15, and the same caveat applies: AI is unreliable on second-order semantics. The eager-vs-lazy step-eval question is one such second-order question the agent didn’t surface; I had to lock it in the contract before the build and the agent followed.

6. Session 12 – PM Agent: Interpreter Commits Review

Source: `session12_pm_interpreter_review_2026-05-21.jsonl`. PM-agent session that reviewed the two developer commits against the runtime contract in `part2_plan.md`.

Entry #21 of 24 • Date 2026-05-21 • **Phase** Post-developer review (PM agent) • **AI tool** Claude Opus 4.7

Goal. Have the PM read the developer’s interpreter + test commits, run the test suite, and produce a realistic review (not a rubber stamp) before shipping the work to İsmail’s typechecker handoff.

Prompt. PM-agent prompt covering: scope (commits 51c1b03 and fac696a only; do not modify code; do not touch `typechecker.py`; produce a written review with file:line citations), the contract sections to verify against, the six rubric dimensions, and a structured deliverable.

Accepted. PM produced a structured review with per-contract verdicts. Three gaps surfaced:

- **Gap F:** Runtime error #4 (substitute of unknown ingredient) had no test pinning the message.
- **Gap G:** Decision #31 (implicit Ingredient → Quantity projection) had implementation coverage but no dedicated test class.
- **Gap I:** AmbiguousCall dispatcher had a fallback that dispatched via symbol-table isinstance checks – silent-failure mode that would corrupt integration testing.

All three gaps were classified as “fix tonight” with concrete remediation steps. I dispatched a second developer-agent round; that round landed as d5c539c and the gaps were all closed. Test count went 147 → 152.

Rejected / modified. Nothing rejected – the three gaps were all real.

Errors caught. The single most useful PM finding was Gap I (silent-AmbiguousCall fallback). It’s the kind of bug that wouldn’t surface until İsmail’s typechecker is fully wired in and tests are running through the full pipeline – at which point a missed AmbiguousCall would corrupt the output. The PM caught it by reading the interpreter source against the contract claim of “loud raise on AmbiguousCall.”

Reflection. Two PM rounds across both parts of this project (entry 8 Part 1, entry 21 Part 2) have produced the same shape of finding: silent-failure modes that would propagate downstream. The PM doesn’t catch obvious bugs – the developer agent has tests for those. The PM catches the kind of bug where two parts of the system look fine in isolation but disagree under integration. *Lesson:* ask the PM to read contracts and implementations *against each other*, not separately.

7. Session 13 – Professor Agent: Pass-1 + Final Rubric Review

Source: `session13_professor_2026-05-21_to_2026-05-23.jsonl`. Same session UUID extended across two days: a Pass-1 narrow-scope review on May 21 and a final full-scope review on May 23 after the type checker and bug-fix rounds had landed. Two entries (22 and 23) cite this one session.

Entry #22 of 24 • Date 2026-05-21 • **Phase** Pass-1 rubric review (autonomous agent) • **AI tool** Claude Opus 4.7

Goal. Get a tenured-PL-professor-style review of the in-scope artifacts under the six-dimension rubric (Correctness, Design Justification, Sebesta Framework, Originality, Code Quality, Exam Performance) before submission, with predicted exam questions for each partner.

Prompt. Professor-agent prompt (~1.2k characters) covering: scope (locked or stable artifacts only – typechecker explicitly out of scope at this point), required output structure (rubric grades, top findings per dimension, predicted exam questions with target answers and exposure ratings, triage list), tone (skeptical, “find what the grader will find”).

Accepted. Agent produced a structured ~2900-word review with: tentative letter grades per rubric dimension (B+ to A– across the six), 14 findings clustered by dimension with file:line citations, 5+5 predicted exam questions with target answers and exposure ratings, and a triage list of fix-tonight / defer / ignore items.

Rejected / modified. Nothing rejected substantively; every finding I cross-checked against the actual code matched the agent’s claim.

Errors caught. F1 (the eager-vs-lazy step evaluation inconsistency surfacing in sample 1’s output) is the kind of finding I would have walked into the exam with unprepared. The grader would have asked “why does sample 1 show serves 6 but only 4 loop iterations?” F4 (error #19 referenced in plan §2.6 but missing from spec §10’s error list) and F5 (no Sebesta hook for why `scale` skips Temperature/Duration) were both fixable in 5 minutes and not caught by the PM review (which was looking for contract-vs-implementation drift, not spec-vs-plan drift).

Reflection. The two-agent pattern I converged on by the end of Part 2 – PM agent for contract-vs-implementation drift, professor agent for grader-vs-submission drift – is the working pattern I’ll take into the next multi-week project. Each agent has a different default failure mode: the PM defaults to “this code passes its tests, looks fine,” the professor defaults to “where would I subtract points.”

Postscript (2026-05-23). The F1 “docstring fix + §4.4 sentence” resolution recorded above was superseded later that week. After İsmail’s bug-fix round introduced a `captured_params` re-execution mechanism for `scale`, I accepted the new semantics (`scale` re-executes step bodies under the rebound `servings`) rather than continuing to defend the eager-only behavior with a docstring patch. The professor pass’s F1 recommendation became obsolete – the better resolution was to take the free win the new implementation offered. D1 §4.4 and D3 §2 were rewritten to match the new semantics. The original docstring drift is now resolved by code matching docs in the opposite direction from what the Pass-1 review proposed.

Entry #23 of 24 • Date 2026-05-23 • **Phase** Final rubric review (autonomous agent, session 13 continuation) • **AI tool** Claude Opus 4.7

Goal. Final pre-submission rubric review of the complete Part 2 submission – typechecker landed, bug-fix round integrated, samples running end-to-end, documents drafted – to confirm SHIP and surface any last fixable findings.

Prompt. Same professor framing as Pass-1, scoped to the finalized state: now includes the typechecker, the D1 [P2] sections, the D3 PDF stub, D5, and the cumulative D4 in progress.

Accepted. Agent produced rubric grades, 5+5 predicted exam questions per partner, three ranked fix-tonight items, and a clean SHIP-WITH-FIXES verdict. Predicted exam questions for me cover the `scale` re-execution rule (**B1**, low exposure – defended in D1 §4.4 and D3 §2); banker’s rounding rationale (**B2**, low exposure – pinned by tests); Sebesta §7.6 operand evaluation order (**B3**, medium exposure – rehearse the C-style contrast on `i++ + i++`); the step-body partial-evaluator in `interpreter.py` (**B4**, low exposure – code is mine); and Sebesta §7.2 precedence of `quantity_of` (**B5**, low exposure – decision #34 rationale in spec). Predicted questions for İsmail cover the type-equivalence split (**I1**, low), `int/float` coercion asymmetry (**I2**, low), the three-class `pour` verdict (**I3**, medium – v2 refinement framing), `AmbiguousCall` rewrite-vs-annotate (**I4**, medium – must have read `interpreter.py`), and Pinch as primitive (**I5**, low).

Rejected / modified. Nothing rejected. Three fix-tonight items ranked by leverage: (F1) edit one sentence in entry 22 to align the journal narrative with the eventual fix path; (F3) add error #19 row to spec §10’s table; (F2) wrap `env.define()` in `_eval_LetStmt` with try/except to catch `RedeclarationError`. All three are ≤ 10 minutes each and none touch tested code paths.

Errors caught. The F1 finding is the single most useful one from this pass: my Pass-1 entry described one fix path (docstring + §4.4 sentence), and the eventual fix path was different (full re-execution rule via `captured_params`). Without the postscript on entry 22 (now added), a grader who reads $D4 \rightarrow D5 \rightarrow \text{code}$ in order would catch the drift in 30 seconds.

Reflection. The same-session-extended-across-days pattern is something I’ll take forward. The professor agent had Pass-1 context already in its working memory; the May 23 continuation didn’t need re-onboarding. The trade-off is that the cumulative output is harder to grep against – there are two reviews layered in one transcript – but for personal use the continuity is worth it. Lesson for the exam: *the professor’s job is to flag drift between artifacts; the dev’s job is to flag drift between code and tests; both jobs are the human’s job to integrate.*

8. Session 14 – Tester Agent: Verification

Source: `session14_tester_2026-05-22_to_2026-05-23.jsonl`. The same session ran two verification turns: a post-handoff integration audit on 2026-05-22 (its own prompt scoped to the just- landed typechecker handoff), and a final pre-submission audit on 2026-05-23 (a fresh prompt scoped to “verify the submission against its own claims”). Entry 24 documents the May 23 turn, which returned the SHIP verdict.

Entry #24 of 24 • Date 2026-05-23 • **Phase** Functional verification (autonomous agent, second turn) • **AI tool** Claude Opus 4.7

Goal. Run the final pre-submission verification pass on the Recipix Part 2 project against `origin/main`. Confirm 152 tests green, all CLI commands behave as documented, sample-3 type-error trace is pinned verbatim, the 19 §10 errors are covered, and the four canonical action-verb test cases produce the locked verdicts.

Prompt. Tester-agent prompt covering: scope (run, don’t modify; no commit, no push), required output structure (test count, CLI results, sample-3 trace verbatim, 19-error coverage table, action-verb verdicts, SHIP verdict), and explicit cleanup of any temporary `.rcx` files created for the action-verb cases.

Accepted. Agent verified commit `db7a241`, ran all six target CLI commands, built the 19-error coverage table, and ran the four action-verb cases via temp `.rcx` files which it then cleaned up.

Verdict: SHIP. Concrete numbers:

- `python -m unittest discover src/tests` → “Ran 152 tests in 0.039s – OK” (0 failures, 0

errors).

- `python main.py -run sample1.rcx → pancakes - serves 6` with the six pour/flip cycles (scale re-executes step bodies per db7a241).
- `python main.py -run sample2.rcx → vegan smoothie` with milk removed by substitute, then non-vegan with both milk and oat_milk shown.
- `python main.py -typecheck sample3.rcx → exit 1, type error at line 13, col 0: dimension mismatch: cannot + Mass and Volume.`
- Imports smoke-test → imports OK.

The 19-error coverage table shows every error has a type-checker code path with file:line citation. Action-verb verdicts: all four canonical cases produce the expected accept/reject with the exact error messages.

Rejected / modified. Nothing rejected – agent did exactly the verification asked for, no scope creep into modifying code or “helpful” refactors. The two minor non-blocking observations the tester surfaced – (a) no dedicated `test_typechecker.py` pinning per-error messages (regression fixtures provide coverage but not regression-proof), and (b) sample 3’s column is reported as 0 rather than the column of + – are both cosmetic.

Errors caught. None – the submission was already clean before this pass. The tester’s contribution was *confirming the SHIP claim with concrete numbers*, which is the verification any submission deserves before clicking upload. The verbatim sample-3 trace this session captured is the one pinned in D3 §4.

Reflection. A functional verification turn before submission is cheap insurance, and reusing the same session across two days kept the agent’s context intact: it didn’t have to re-read the contract, the spec, and the plan from scratch on the second turn. The report adds nothing the test suite doesn’t already say, but the formatting – exit codes, first 15 lines of output, the four action-verb temp programs run independently of the existing test fixtures – is exactly the artifact I want sitting alongside the submission as proof-of-shippability. *Lesson for next time:* schedule the verification turn on a frozen tree where the only remaining action is *click upload*, not on a tree still being edited.

Cumulative summary

Across Part 1 (15 entries from 6 sessions, May 6–8) and Part 2 (this continuation, 9 entries from 8 sessions, May 20–23), the journal documents **14 substantive AI sessions producing 24 entries**. The breakdown:

- **6 Claude.ai web sessions** – 2 brainstorming (Part 1 sessions 1 and 2), 1 Part 2 planning (session 7), 3 E-experiments E1/E2/E3 (sessions 8, 9, 10).
- **2 Claude Code multi-turn PM threads** – both in Part 1 (sessions 3 and 4 – Sonnet 4.6 and Opus 4.7 respectively, multi-turn user-driven orchestration).
- **6 Claude Code autonomous-agent runs** – Part 1 contributes 2 (tester agent session 5 + developer agent session 6 across two rounds); Part 2 contributes 4 (developer session 11, PM session 12, professor session 13 across two passes, tester session 14).

Mandatory experiments E1 (catch-up – see entry 17 note), E2, and E3 are all documented with shareable `claude.ai/share` URLs at sessions 8, 9, 10. Four Claude Code transcripts are uploaded as JSONL files alongside this PDF for sessions 11, 12, 13, 14.

The journal contains **seven documented cases** where the AI was wrong, suboptimal, or made claims I had to verify against the spec, the repo, or Sebesta and override:

- Entry 2 (Part 1): structural equivalence suggested for **Recipe** type; caught by reading Sebesta §6.14 against the suggestion.
- Entry 8 (Part 1): two parallel `tokens.py` files duck-typed compatibly but would silently break under rename; caught by the PM agent.
- Entry 14 (Part 1): developer agent generalized decision #17 too broadly; caught by the tester-agent's `a < b == c` fixture.
- Entry 17 (Part 2 / E1): `substitute` slots typed as **Expression** instead of bare **IDENT**; caught by reading spec §13 against the AI's grammar.
- Entry 18 (Part 2 / E2): name equivalence recommended for **Ingredient** based on hypothetical record collisions that a closed `v1` forecloses; caught by reading spec §1 and decision #10 against the recommendation.
- Entry 19 (Part 2 / E3): 12-row per-verb action-verb signature table instead of the 3-class collapse; caught by reading plan §2.3 against the AI's implementation.
- Entry 22/23 (Part 2 / Professor): F1 scale-vs-loop inconsistency in the Pass-1 review; the recommended fix (docstring patch) was superseded by a better fix (`captured_params` re-execution rule) when İsmail's bug-fix round arrived. Caught by re-reading entry 22 in light of the May 22–23 changes.

That's seven documented AI-was-wrong moments across both parts of the project – above the “three documented cases” floor the handout sets, and weighted toward the design / type-system corners where the rubric's Design Justification and Sebesta Framework dimensions are graded.

End of Part 2 continuation. Combined with the Part 1 journal (210104004132_D4_P1.pdf, May 8 submission), this completes the D4 deliverable for CSE 341 Spring 2026.